

Pointers in C → Day 1

1. Introduction to Pointers



1.1 What is a Pointer?

A pointer in C is a variable that stores the **memory address** of another variable.

It allows indirect access and manipulation of data in memory, a critical feature for operating systems.

```
int a = 10;  
int *p = &a;
```

Here, `p` stores the address of variable `a`, i.e., `p` points to `a`.

1.2 Why are Pointers Important?

- **Memory Efficiency:** You can manipulate large data structures like arrays or structs by passing addresses instead of copying entire values.

- **Dynamic Memory:** Enables usage of `malloc`, `calloc`, `realloc`, and `free`.
 - **Data Structures:** Pointers are the backbone of linked lists, trees, graphs.
 - **Function Arguments:** Enables call by reference.
 - **System Programming:** Used for manipulating hardware, I/O operations, and memory-mapped devices.
 - **The Gate to Segmentation Faults:** Mishandled pointers are the most common source of crashes in C, making pointer literacy essential.
-

2. Referencing and Dereferencing

2.1 The `&` Operator → Referencing

`&` is the address-of operator. It gives the memory address of a variable.

```
int x = 42;  
int *p = &x; // p now holds the address of x
```

2.2 The `*` Operator → Dereferencing

`*` is the dereference operator. It accesses the value stored at the memory address.

```
printf("%d\n", *p); // prints 42
```

2.3 Overall Picture

Input

```
#include <stdio.h>  
  
int main() {  
  
    int x = 10;  
    int *ip = &x;
```

```

// Same value is printed
printf("%d\n",x);
printf("%d\n\n",*ip);

// Same value is printed
printf("%p\n",ip);
printf("%p\n\n",&x);

printf("%p\n",&ip);

return 0;
}

```

Output

```

10
10

0x7ffdb8b2dcec
0x7ffdb8b2dcec

0x7ffdb8b2dce0

```

Operation	Pointer	Normal Variable
Obtain Value	Yes	Yes
Obtain address using &	Yes	Yes
Dereference using *	Yes	No

3. Pointer Arithmetic

A general overview of pointer arithmetic

```

datatype *p = &a;
p + n = &a + n * sizeof(datatype);

```

Pointer arithmetic is scaled by the size of the data type →

```
int arr[] = {10, 20, 30};  
int *p = arr;  
p + 1 // points to arr[1], i.e., 4 bytes forward (if int is 4 bytes)
```

3.1 Increment and Decrement

```
p++; // moves pointer to next element  
p--; // moves pointer to previous element
```

3.2 Pointer Difference

```
int diff = &arr[2] - &arr[0]; // yields 2 (not byte difference, but element count)
```

3.3 Comparison

```
if (p1 < p2) // safe only if p1 and p2 point into the same array
```

3.4 Quiz Time

Question →

```
#include <stdio.h>  
  
int main() {  
  
    unsigned int x = 3251;  
    char *cp = &x;  
    unsigned char c1 = *cp++, c2 = *cp;  
    printf("%d\n", c1);  
    printf("%d\n", c2);  
  
    return 0;  
}
```

4. Precedence Rule

4.1 Let's Understand

```
*p++    // increments pointer, then dereferences old location
*(p++)  // same as *p++, just clearer syntax
(*p)++  // increments value at pointer location
```

4.2 Example

```
int arr[] = {1, 2, 3};
int *p = arr;

printf("%d\n", *p++); // 1, then p points to 2
printf("%d\n", (*p)++); // 2, then arr[1] becomes 3
```

5. **void *** → The Chameleon of Pointers



5.1 Nature of **void ***

The `void *` pointer, often called the *generic pointer*, is a powerful feature in C. It's used when the type of data the pointer will point to is not known at the time of writing code. This enables **generic programming**, such as writing data structure libraries, memory allocators, and system-level interfaces.

Unlike other pointers, a `void *` doesn't have a data type associated with it. That means it doesn't know how many bytes it refers to, so →

- You **can assign** any type's address to a `void *`.
- You **cannot dereference** a `void *` without typecasting it first.
- You **cannot perform pointer arithmetic** on `void *` in standard C.

5.2 Why use `void *` ?

- **Memory allocation:** Functions like `malloc()` return a `void *`, which can be assigned to any pointer type.
- **Generic functions:** Standard library functions accept `void *` to operate on any type of data.
- **Function parameters:** When building polymorphic behavior using callbacks or event handlers.

5.3 Example: Assigning and Casting

```
void *p;
int a = 99;
p = &a;

// Compiler error: can't dereference void * directly
// printf("%d\n", *p); // ❌

// Correct: cast before dereferencing
printf("%d\n", *(int *)p); // ✅
```

The `(int *)` cast tells the compiler to treat `p` as if it were pointing to an `int`, so it knows to dereference 4 bytes (on a 32-bit system).

5.4 Casting with Other Types

```
float pi = 3.14;
void *vp = &pi;
printf("%f\n", *(float *)vp);

char c = 'Z';
vp = &c;
printf("%c\n", *(char *)vp);
```

5.5 Using `void *` with Malloc

```
int *arr = (int *)malloc(5 * sizeof(int));
```

Here, `malloc()` returns a `void *`, and we cast it to `int *` to store integers.

5.6 Building a Generic Swap Function

```
void swap(void *a, void *b, size_t size) {
    char *pa = (char *)a;
    char *pb = (char *)b;
    for (size_t i = 0; i < size; i++) {
        char temp = pa[i];
        pa[i] = pb[i];
        pb[i] = temp;
    }
}
```

Usage:

```
int x = 10, y = 20;
swap(&x, &y, sizeof(int));

float f1 = 3.14, f2 = 2.71;
swap(&f1, &f2, sizeof(float));
```

This kind of code is foundational for writing standard library implementations.

5.7 What You Cannot Do with `void *`

```
void *p = malloc(sizeof(int));  
*p = 10; // ❌ Error: compiler doesn't know the type  
*(int *)p = 10; // ✅ Correct: explicitly cast to int *  
  
p++; // ❌ Invalid: pointer arithmetic on void * is not allowed in standard C
```

6. Function Pointers

6.1 What Is a Function Pointer?

You already know what a function is:

```
int add(int a, int b) {  
    return a + b;  
}
```

You call it like this:

```
int result = add(2, 3);
```

So what is a **function pointer**?

A function pointer is a **variable that stores the address of a function**.

Just like you can store the address of an

`int` in an `int*`, you can store the address of a function in a function pointer.

6.2 Why Not Just Use Regular Functions?

Choose Function at Runtime (Event Driven Programming)

You don't always know which function to call until your program is running. Function pointers let you pick one **dynamically**, just like picking a variable.

Reusability via Callbacks

You can pass function pointers into other functions as **arguments** to create reusable, generic behavior (like event handlers or comparators).

Dispatch Tables

Arrays of function pointers can simulate dynamic dispatch or function selection, which is crucial in systems like OS kernels and interpreters.

Extensibility

Plugins and drivers can register their behavior via function pointers without changing the core logic.

6.3 Syntax and Examples

Step 1: Declare a Function

```
int add(int a, int b) {  
    return a + b;  
}
```

Step 2: Declare a Function Pointer

```
int (*fptr)(int, int);
```

This means: fptr is a pointer to a function taking two int arguments and returning an int.

Step 3: Assign the Function to the Pointer

```
fptr = add; // or fptr = &add;
```

Step 4: Call the Function Through the Pointer

```
int result = fptr(5, 3); // works exactly like add(5, 3)
```

7. Double Pointers

Before jumping to 2D arrays, let's first understand how a dynamic array grows.

Simple Dynamic Array Example →

```
#include <stdio.h>
#include <stdlib.h>

int main() {
    int *arr = NULL;
    int capacity = 2; // start small
    int size = 0;
    arr = malloc(capacity * sizeof(int));

    for (int i = 0; i < 10; i++) {
        if (size == capacity) {
            capacity *= 2; // double the size
            arr = realloc(arr, capacity * sizeof(int));
        }
        arr[size++] = i * 10; // insert element
    }

    for (int i = 0; i < size; i++) {
        printf("%d ", arr[i]);
    }

    free(arr);
    return 0;
}
```

This pattern is the backbone of `std::vector` in C++ and dynamic lists in other languages.

7.1 What is a Double Pointer?

A **double pointer** is a pointer that stores the address of another pointer. Think of it as a "pointer to pointer."

```
int a = 10;
int *p = &a;
int **pp = &p;

printf("%d\n", **pp); // prints 10
```

Here's what's happening:

- `a` is an integer
- `p` is a pointer to `a`
- `pp` is a pointer to `p`
- `pp` gives `p`
- `*pp` gives the value of `a`

7.2 Why Use Double Pointers?

Use Case 1: Dynamic 2D Arrays

```
int **matrix = malloc(rows * sizeof(int*));
for (int i = 0; i < rows; i++) {
    matrix[i] = malloc(cols * sizeof(int));
}
```

Use Case 2: Modifying a Pointer in a Function

```
void allocate(int **ptr) {
    *ptr = malloc(sizeof(int));
    **ptr = 42;
}
```

Use Case 3: Arrays of Strings

```
char *arr[] = {"hello", "world"};
char **p = arr;
printf("%s\n", *p); // prints "hello"
```

7.3. Understanding `char*`, `char[]`, `char**`, and `void**`

What is a String in C?

In C, a **string** is just an array of characters **terminated by a null character** (`'\0'`). This null character marks the end of the string. It's what differentiates →

- A character array: just a list of characters
- A C string: a null-terminated character array

```
char name[] = {'R','a','c','h','i','t'}  
char name1[] = "Rachit"; // char array with null terminator (7 bytes)  
char *name2 = "Rachit"; // pointer to a string literal with '\0' at the end
```

If you forget to include `'\0'`, many string functions like `printf("%s", str)`, `strlen()`, `strcpy()` will keep reading memory until they accidentally hit a `'\0'`, which is undefined behavior.

Visual Representation of name1 →

```
name1[] = {'R','a','c','h','i','t','\0'}
```

Differences Between `char[]` and `char*`

Feature	<code>char[]</code>	<code>char*</code>
Memory Location	Allocated on stack	Points to literal or heap
Mutability	Can be modified	Literal version is read-only
Size	Includes <code>'\0'</code> , size fixed	Just pointer to first char
Terminator	Must end with <code>'\0'</code>	Must be <code>'\0'</code> -terminated manually

Example:

```
char str1[] = "hello"; // modifiable, lives on stack  
char *str2 = "hello"; // read-only, lives in text segment
```

Trying to modify `str2[0] = 'H';` causes undefined behavior, but `str1[0] = 'H';` works fine.

`char*` → A Pointer to a Character

This is used to represent strings in C →

```
char *str = "hello";  
printf("%s\n", str);
```

`str` points to the first character of a null-terminated array of characters.

`char**` → Pointer to a String Array

This can represent multiple strings, like `argv` in `main` →

```
int main(int argc, char **argv) {  
    printf("Program name: %s\n", argv[0]);  
}
```

Here, `argv` is a pointer to an array of `char*`, and `argv[0]` is the first string.

`void**` → Generic Pointer to Pointer

Useful in APIs that modify data of unknown type →

```
void allocate(void **p, size_t size) {  
    *p = malloc(size);  
}  
  
int *arr;  
allocate((void**)&arr, 10 * sizeof(int));
```

This is the generic equivalent of `int**`, `float**`, etc.

Caution →

- Always match the level of indirection with the usage.
- Misuse leads to undefined behavior or segmentation faults.